

VERIFICATION PLAN

AXI-Lite Slave IP with Memory

ECE-593: Fundamentals of Pre-Si Validation

Portland State University

Team: Patrick, Pratibha

Document Information

<i>Field</i>	<i>Value</i>
<i>Document ID</i>	<i>VPL-AXIL-001</i>
<i>Project / Product</i>	<i>AXI-Lite Slave IP with Memory (axi_lite_mem)</i>
<i>Design Version</i>	<i>RTL v0.1 (MS1 baseline)</i>
<i>Verification Level</i>	<i>Block-level</i>
<i>Prepared By</i>	<i>Patrick, Pratibha (Verification — shared ownership)</i>
<i>Reviewed By</i>	<i>Patrick, Pratibha</i>

<i>Approved By</i>	<i>Instructor — Venkatesh Patil</i>
<i>Date Created</i>	<i>April 14, 2026</i>
<i>Last Revised</i>	<i>April 20, 2026</i>
<i>Revision</i>	<i>1.0 – Initial Release</i>
<i>Status</i>	<i>Draft</i>
<i>Confidentiality</i>	<i>Internal (coursework)</i>

Purpose Statement

This Verification Plan defines the strategy, environment, tools, and completion criteria for the functional verification of the AXI-Lite Slave IP with Memory. It is structured in accordance with the hierarchical verification methodology described in Comprehensive Functional Verification by Bruce Wilde.

For ECE-593: Fundamentals of Pre-Si Validation, instructed by Venkatesh Patil.

Git Repository

https://github.com/a215364731/ece593_final_project

All deliverables for each milestone live under the corresponding subdirectory:

*AXI_Lite_MS1_Trad/ AXI_Lite_MS2_Class/ AXI_Lite_MS3_Class/
 AXI_Lite_MS4_UVM/ AXI_Lite_MS5_UVM/ AXI_Lite_FINAL/*

1. Introduction

1.1 Purpose and Scope

This Verification Plan (VP) describes the complete approach for functionally verifying the AXI-Lite Slave IP with Memory. It establishes the verification scope, methods, environment, required tools, completion criteria, and resource plan.

This document applies to the following verification level:

- ☒ Block-level verification
- ☐ Sub-system verification
- ☐ Full-chip / Top-level verification
- ☐ System-level verification

1.2 Design Overview

The Design Under Verification (DUV) is a parameterized AXI4-Lite slave that wraps an internal dual-port block RAM. The top-level module `s_axil_top` instantiates two submodules — `s_axil_wr` (handling AW, W, and B channels) and `s_axil_rd` (handling AR and R channels) — and a shared memory array of `MEM_DEPTH` words, each `DATA_WIDTH` bits wide. All five AXI4-Lite channels follow the standard VALID/READY handshake and the design supports byte-enable writes via `WSTRB`, out-of-range `SLVERR` responses, and write-before-read forwarding when a read and write target the same word in the same cycle.

Refer to HLDS-AXIL-001 (Design Specification) for the complete block diagram, port list, parameter definitions, and channel behavior.

1.3 Verification Hierarchy

Verification of this design follows a hierarchical approach consistent with Bruce Wilde's Comprehensive Functional Verification framework. Each level of the hierarchy addresses a specific set of functions and feeds results to the next level.

Hierarchy Level	Design Scope	Covered In This VP	Referenced VP
<i>Block</i>	<i>s_axil_top (with wr, rd, memory)</i>	<i>☒ Yes</i>	<i>—</i>
<i>Sub-system</i>	<i>AXI interconnect + slaves</i>	<i>☐ No</i>	<i>Future VP</i>
<i>Full-chip</i>	<i>SoC integration</i>	<i>☐ No</i>	<i>Future VP</i>
<i>System</i>	<i>Multi-processor system (ECE-593 FP)</i>	<i>☐ No</i>	<i>VP-SYS-001</i>

1.4 References

Ref #	Document Title	Version	Date
[R01]	HLDS-AXIL-001 – AXI-Lite Slave IP with Memory Specification	1.0	2026-04-20
[R02]	Comprehensive Functional Verification – Bruce Wilde	—	—
[R03]	AMBA AXI and ACE Protocol Specification (ARM IHI 0022E)	E	—
[R04]	ECE-593 Final Project Handout – V. Patil	—	2026
[R05]	SystemVerilog IEEE 1800-2017 LRM	2017	2017

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
<i>DUV</i>	<i>Design Under Verification</i>
<i>DUT</i>	<i>Device Under Test</i>
<i>UVM</i>	<i>Universal Verification Methodology</i>
<i>SV</i>	<i>SystemVerilog</i>
<i>TB</i>	<i>Testbench</i>

<i>FC</i>	<i>Functional Coverage</i>
<i>CC</i>	<i>Code Coverage</i>
<i>CR</i>	<i>Coverage Report</i>
<i>VP</i>	<i>Verification Plan</i>
<i>AXI</i>	<i>Advanced eXtensible Interface</i>
<i>WSTRB</i>	<i>Write byte-lane strobe signal</i>
<i>BRESP / RRESP</i>	<i>Write / Read response (2-bit AXI encoding)</i>
<i>SLVERR</i>	<i>Slave error response (RESP = 2'b10)</i>
<i>SVA</i>	<i>SystemVerilog Assertions</i>
<i>RAL</i>	<i>Register Abstraction Layer (UVM)</i>
<i>MS</i>	<i>Milestone (MS1 through MS5 + FINAL)</i>

2. Functions to Be Verified

This section lists all functions and features of the DUV that are within scope of this verification plan, as required by the verification hierarchy (Wilde, Chapters 3–4). Each function is assigned a unique identifier for traceability.

<i>FV-ID</i>	<i>Function / Feature Description</i>	<i>Design Ref</i>	<i>Priority</i>	<i>Verification Method</i>
<i>FV-001</i>	<i>Reset behavior — all control flags and pipeline registers initialize to defined values on resetn assertion</i>	<i>DS §5 Reset</i>	<i>Critical</i>	<i>Directed + Assertions</i>
<i>FV-002</i>	<i>AW channel VALID/READY handshake — AWREADY gating by aw_active</i>	<i>DS §7 AW</i>	<i>Critical</i>	<i>Directed + SVA + FC</i>
<i>FV-003</i>	<i>W channel VALID/READY handshake — WREADY gating by w_active</i>	<i>DS §7 W</i>	<i>Critical</i>	<i>Directed + SVA + FC</i>
<i>FV-004</i>	<i>Write commit — both AW and W must latch before memory update (order-independent)</i>	<i>DS §7 Commit</i>	<i>Critical</i>	<i>Directed (AW-first, W-first, concurrent)</i>
<i>FV-005</i>	<i>B channel response — BVALID / BRESP / BREADY timing for OKAY and SLVERR</i>	<i>DS §7 B</i>	<i>Critical</i>	<i>Directed + SVA + Scoreboard</i>
<i>FV-006</i>	<i>WSTRB byte-lane masking — only enabled bytes update memory; others preserved</i>	<i>DS §4 Write Path</i>	<i>High</i>	<i>Directed (all strobe patterns) + FC</i>
<i>FV-007</i>	<i>Out-of-range write — mem_w_en stays low; BRESP = SLVERR</i>	<i>DS §7 Commit</i>	<i>Critical</i>	<i>Directed + FC</i>
<i>FV-008</i>	<i>AR channel VALID/READY handshake — ARREADY gating by in-flight read</i>	<i>DS §7 AR</i>	<i>Critical</i>	<i>Directed + SVA + FC</i>

FV-009	<i>R channel response — RVALID / RDATA / RRESP timing and data correctness</i>	<i>DS §7 R</i>	<i>Critical</i>	<i>Scoreboard</i>
FV-010	<i>Out-of-range read — RDATA = 0, RRESP = SLVERR</i>	<i>DS §7 R</i>	<i>Critical</i>	<i>Directed + FC</i>
FV-011	<i>Write-before-read forwarding — same-cycle same-address WR/RD returns WSTRB-merged data</i>	<i>DS §4 Forwarding</i>	<i>High</i>	<i>Directed + Scoreboard</i>
FV-012	<i>Data integrity — every written value read back matches under all strobe patterns</i>	<i>DS §4</i>	<i>Critical</i>	<i>Constrained-Random + Scoreboard</i>
FV-013	<i>Backpressure tolerance — slave correctly stalls when master withholds BREADY / RREADY</i>	<i>DS §7</i>	<i>High</i>	<i>Directed + Constrained-Random</i>
FV-014	<i>Back-to-back transactions — no data loss when transactions issue every cycle</i>	<i>DS §7</i>	<i>High</i>	<i>Constrained-Random + Scoreboard</i>
FV-015	<i>Parameter scalability — DUV functions correctly at DATA_WIDTH $\in$ {32, 64} and varying MEM_DEPTH</i>	<i>DS §3 Parameters</i>	<i>Medium</i>	<i>Parameterized regression</i>
FV-016	<i>MEM_INIT preload — memory loads from hex file when MEM_INIT is non-empty</i>	<i>DS §5 Reset</i>	<i>Low</i>	<i>Directed</i>
FV-017	<i>AWPROT / ARPROT acceptance — slave accepts but does not reject any PROT encoding</i>	<i>DS §6 Ports</i>	<i>Low</i>	<i>Directed</i>

Priority legend: Critical = must pass before MS sign-off; High = required for sign-off; Medium = best-effort; Low = stretch goal.

3. Functions Not Covered at This Verification Level

The following functions are explicitly excluded from this plan because they will be verified at a different level of the verification hierarchy or are outside the scope of the AXI-Lite subset.

NC-ID	Function / Feature	Rationale for Exclusion	Verified In
NC-001	AXI4 burst transactions (INCR, WRAP, FIXED)	AXI-Lite restricts to single-beat transfers per the AMBA spec	N/A
NC-002	Exclusive access (AxLOCK, EXOKAY)	Not supported by AXI-Lite subset	N/A
NC-003	AxCACHE and AxQOS	Optional attributes not included in AXI-Lite core	N/A
NC-004	Decode errors (DECERR)	Address decode performed upstream in an interconnect, not by slave	Interconnect VP
NC-005	Clock domain crossing / metastability	Single-clock design; no asynchronous boundaries	N/A
NC-006	System-level bus contention and arbitration	Block-level VP; arbitration verified at sub-system level	Sub-system VP
NC-007	Gate-level timing and STA	Pre-silicon functional scope only; post-synthesis outside scope	STA flow

NC-008	Physical design, power, and DFT	Functional verification scope only	P&R / DFT flows
--------	---------------------------------	------------------------------------	-----------------

4. Verification Strategy and Methods

This section defines the overall strategy and the specific test and checking methods that the verification team will employ, consistent with Wilde Chapters 3–4 and 7–12. The strategy evolves across milestones from a simple directed testbench to a full UVM environment.

4.1 Overall Strategy

The verification effort is organized into five milestones that progressively increase rigor and automation:

Milestone	Methodology	Primary Goals
<i>MS1 – Traditional TB</i>	<i>Task-based module TB in SystemVerilog</i>	<i>Smoke-test basic read/write; confirm RTL compiles clean</i>
<i>MS2 – Class-Based TB</i>	<i>OOP SV classes (transaction, driver, monitor, scoreboard)</i>	<i>Layered TB; self-checking; scoreboard comparison</i>
<i>MS3 – Class-Based + CR</i>	<i>MS2 + constrained-random and functional coverage</i>	<i>Stimulus diversity; coverage-driven iteration; SVA</i>
<i>MS4 – UVM (Basic)</i>	<i>UVM 1.2 agent / env / sequencer / test</i>	<i>Full UVM structure; factory overrides; sequences</i>
<i>MS5 – UVM (Advanced)</i>	<i>MS4 + UVM RAL, layered sequences, regression</i>	<i>Register model; coverage closure; regression signoff</i>

The overall strategy combines:

- *Directed tests — to verify specific corner cases and compliance requirements.*

- *Constrained-random stimulus* — to explore unanticipated scenarios and achieve broad coverage (MS3 onward).
- *Assertion-based verification (ABV)* — to catch protocol violations at the point of occurrence.
 - *Functional coverage* — to measure completeness of the constrained-random campaign.
 - *Code coverage* — to identify unreachable HDL constructs.
- *Formal verification* — applied selectively to FSM reachability and reset correctness (stretch goal).

4.2 Testbench Architecture

The MS1 testbench is a single-module, task-based testbench instantiating the DUT directly and driving all five AXI channels via `write_transaction` and `read_transaction` tasks. Starting at MS2, the testbench refactors into a class-based layered architecture. At MS4 the architecture transitions to UVM 1.2 with a standard agent/env/test hierarchy. A block diagram of the MS4+ testbench architecture is provided in the MS4 deliverables.

4.2.1 Testbench Components

Component	Description	Introduced In	Reuse / New
Top-level TB	Instantiates DUT, clock/reset, interface bindings	MS1	New
AXI-Lite Interface	SV interface grouping the five channels + modport	MS2	New
Transaction class	<code>axi_lite_txn</code> — <code>addr</code> , <code>data</code> , <code>strb</code> , <code>read/write</code> , <code>resp</code>	MS2	New
Driver	Drives AW/W/AR channels, collects B/R responses	MS2	New
Monitor	Samples all channels; publishes observed txns	MS2	New
Scoreboard	Reference memory model; compares observed vs expected	MS2	New

<i>Coverage collector</i>	<i>Functional covergroups (address, strobe, resp, XR)</i>	<i>MS3</i>	<i>New</i>
<i>Assertion library</i>	<i>SVA properties bound to DUT interfaces</i>	<i>MS3</i>	<i>New</i>
<i>UVM Agent</i>	<i>Packages driver + monitor + sequencer</i>	<i>MS4</i>	<i>New</i>
<i>UVM Env</i>	<i>Assembles agent, scoreboard, coverage</i>	<i>MS4</i>	<i>New</i>
<i>UVM Test(s)</i>	<i>Top-level tests extending uvm_test</i>	<i>MS4</i>	<i>New</i>
<i>UVM RAL / Reg Model</i>	<i>UVM register model of the memory</i>	<i>MS5</i>	<i>New</i>

4.3 Stimulus Methods

4.3.1 Directed Tests

Directed tests deterministically exercise specific DUT behaviors. The directed test matrix for the AXI-Lite slave includes:

- *Reset: assert / de-assert with in-flight transactions active*
- *Single write then single read at same address (data integrity)*
 - *Walking-ones and walking-zeros data patterns*
- *All WSTRB patterns (0000, 0001 ... 1111 for DATA_WIDTH=32)*
- *In-range / boundary / out-of-range addresses (0x000, 0x3FC, 0x400)*
 - *Back-to-back writes; back-to-back reads*
 - *AW before W; W before AW; concurrent AW+W*
 - *Slow BREADY / slow RREADY backpressure*
- *Write-before-read forwarding: WR and RD same cycle same address*

4.3.2 Constrained-Random Tests

Introduced in MS3. Randomized AXI-Lite transactions with protocol-legal constraints:

- *Random read/write with weighted address bins (90% in-range, 10% out-of-range)*

- *Random WSTRB with coverage-driven weighting toward under-covered patterns*
 - *Random inter-transaction delays (0–15 cycles)*
- *Random BREADY / RREADY assertion delays to stress backpressure paths*
 - *Weighted coverage-driven generation for under-covered functional bins*

4.3.3 Assertion-Based Verification

Introduced in MS3. Key SVA properties (bound to the DUT interface):

- *VALID stability — once VALID is asserted, it remains high until READY is seen*
 - *Payload stability — address/data/strb are stable while VALID is asserted*
 - *Reset behavior — no VALID signals assert while resetn is low*
- *BRESP / RRESP legal encoding — RESP must be OKAY or SLVERR (never EXOKAY/DECERR)*
- *No-X on outputs — RDATA, BRESP, RRESP never X during handshake*
- *B response follows write — every write commit eventually produces a BVALID*
- *R response follows AR — every AR handshake eventually produces RVALID*

4.3.4 Formal Verification

Stretch goal. Candidate formal targets:

- *FSM reachability — all states of wr and rd submodules are reachable*
- *Deadlock freedom — no combination of backpressure can stall the slave permanently*
- *Reset completeness — every sequential element reaches its reset value in one clock*

4.4 Checking Strategy

Result checking uses the following methods:

- *Scoreboard-based checking — TB maintains a reference memory model; every read response is compared against the expected value computed from the sequence of writes (with WSTRB merging).*
- *In-line assertion checking — SVA properties fire immediately on protocol violations.*
- *Response-code checking — every BRESP and RRESP is compared against the expected OKAY/SLVERR based on address range.*
- *End-of-simulation report — summary of pass/fail counts, assertion statistics, and coverage percentages.*

5. Verification Environment

5.1 Simulation Environment

Field	Value
<i>Simulator</i>	<i>Synopsys VCS</i>
<i>Simulator Version</i>	<i>VCS 2023.03 or later (PSU lab installation)</i>
<i>Verification Methodology</i>	<i>Traditional (MS1) → Class-based (MS2-3) → UVM 1.2 (MS4-5)</i>
<i>HDL Language</i>	<i>SystemVerilog IEEE 1800-2017</i>
<i>Assertion Language</i>	<i>SystemVerilog Assertions (SVA)</i>
<i>Compile Options</i>	<i>-sverilog -ntb_opts uvm-1.2 -timescale=1ns/1ps +define+UVM_NO_DEPRECATED -full64</i>
<i>Coverage Options</i>	<i>-cm line+cond+fsm+tgl+branch+assert -cm_dir ./cov.vdb</i>
<i>Sim / Runtime Options</i>	<i>+vcs+lic+wait +fsdb+autoflush -assert svaext</i>
<i>Waveform Format</i>	<i>FSDB (Verdi) — \$fsdbDumpvars already present in TB</i>
<i>Seed Management</i>	<i>+ntb_random_seed=<N> for regression; +ntb_random_seed_automatic for exploration</i>
<i>Regression Framework</i>	<i>Makefile-driven; targets: compile, sim, cov, verdi, clean</i>

5.2 Coverage Plan

Functional coverage groups map directly to the FV-IDs in Section 2. Targets are set for MS3 onward; MS1/MS2 track code coverage only.

<i>FV-ID(s)</i>	<i>Coverage Group</i>	<i>Bins / Cross Points</i>	<i>Target</i>	<i>Tool</i>
<i>FV-001</i>	<i>cg_reset</i>	<i>resetrn assertion × control state = default</i>	<i>100%</i>	<i>VCS</i>
<i>FV-002/003</i>	<i>cg_aw_w_hs</i>	<i>AW/W handshake order: AW-first, W-first, concurrent</i>	<i>100%</i>	<i>VCS</i>
<i>FV-005</i>	<i>cg_bresp</i>	<i>BRESP ∈ {OKAY, SLVERR} × BREADY delay bins {0, 1-3, 4-15}</i>	<i>100%</i>	<i>VCS</i>
<i>FV-006</i>	<i>cg_wstrb</i>	<i>WSTRB values 0001 ... 1111 (all 15 non-zero patterns)</i>	<i>100%</i>	<i>VCS</i>
<i>FV-007/010</i>	<i>cg_oor</i>	<i>Out-of-range address × op ∈ {WR, RD} × response</i>	<i>100%</i>	<i>VCS</i>
<i>FV-008</i>	<i>cg_ar_hs</i>	<i>ARREADY gating × RREADY delay bins</i>	<i>100%</i>	<i>VCS</i>
<i>FV-009</i>	<i>cg_rresp</i>	<i>RRESP ∈ {OKAY, SLVERR} × in-range × boundary</i>	<i>100%</i>	<i>VCS</i>
<i>FV-011</i>	<i>cg_fwd</i>	<i>WR + RD same cycle × address match × WSTRB pattern</i>	<i>95%</i>	<i>VCS</i>
<i>FV-012</i>	<i>cg_addr</i>	<i>Word addresses binned across [0, MEM_DEPTH-1]</i>	<i>90%</i>	<i>VCS</i>
<i>FV-013</i>	<i>cg_backpressure</i>	<i>BREADY/RREADY stall cycles × transaction type</i>	<i>90%</i>	<i>VCS</i>

FV-014	cg_b2b	Back-to-back WR-WR, RD-RD, WR-RD, RD-WR	100%	VCS
--------	--------	--	------	-----

6. Required Tools

This section lists all software tools required to support the described verification environment, consistent with Wilde Chapters 5–6.

Category	Tool / Package	Version	Vendor / Source	Status
Simulator	Synopsys VCS	2023.03+	Synopsys	[X] Licensed (PSU lab)
Coverage / URG	VCS Coverage + URG	2023.03+	Synopsys	[X] Bundled with VCS
Waveform Viewer	Synopsys Verdi (FSDB)	2023.03+	Synopsys	[X] Licensed
Linting	Spyglass Lint (optional)	—	Synopsys	[] Optional
Formal Tool	Not planned for MS1–MS5	—	—	[] N/A
VIP	None (hand-coded AXI-Lite driver)	—	Internal	[X] In-house
Regression	GNU Make	≥ 4.2	GNU	[X] Available
Version Control	Git / GitHub	≥ 2.30	GitHub	[X] Available
Bug Tracking	GitHub Issues	—	GitHub	[X] Available

<i>Documentation</i>	<i>MS Word / PDF / Markdown</i>	—	—	<i>[X] Available</i>
----------------------	-------------------------------------	---	---	----------------------

7. Completion Criteria

Verification is declared complete when ALL of the following measurable criteria have been satisfied. Each criterion must be signed off by the team before submission of each milestone.

7.1 Simulation Completion Criteria

#	Criterion	Target Threshold	Applies To
1	<i>All directed tests pass with zero unexpected failures</i>	<i>100% pass rate</i>	<i>MS1–MS5</i>
2	<i>Constrained-random regression passes across N seeds</i>	<i>$N \geq 50$ seeds</i>	<i>MS3–MS5</i>
3	<i>All functional coverage groups close to specified targets</i>	<i>$\geq 90\%$ avg, 100% critical</i>	<i>MS3–MS5</i>
4	<i>Statement and branch code coverage achieved</i>	<i>$\geq 90\%$</i>	<i>MS2–MS5</i>
5	<i>Toggle coverage achieved on DUT outputs and interfaces</i>	<i>$\geq 85\%$</i>	<i>MS3–MS5</i>
6	<i>FSM state and transition coverage achieved</i>	<i>100% reachable</i>	<i>MS3–MS5</i>

7	<i>All SVA assertions pass (zero failures in final regression)</i>	<i>0 failures</i>	<i>MS3–MS5</i>
8	<i>All open bugs triaged; no Critical or High severity bugs outstanding</i>	<i>0 Critical / High open</i>	<i>All MS</i>
9	<i>Waived coverage items documented and approved</i>	<i>Approval on record</i>	<i>MS3–MS5</i>

7.2 Formal Verification Completion Criteria

Formal verification is a stretch goal (not required for MS1–MS5). Criteria apply only if formal is pursued.

#	Criterion	Target	Status
1	<i>All formal properties proven or bounded-proven</i>	<i>PROVEN / BOUNDED</i>	<i>[] Stretch goal</i>
2	<i>No vacuous passes (vacuity checks passed)</i>	<i>0 vacuous</i>	<i>[] Stretch goal</i>
3	<i>All counterexamples analyzed and resolved</i>	<i>0 open CEX</i>	<i>[] Stretch goal</i>

7.3 Sign-Off Checklist

Sign-Off Item	Responsible Party	Date	Sign
<i>MS1 directed TB runs clean on VCS</i>	<i>Patrick, Pratibha</i>	<i>04/20/2026</i>	<i>—</i>
<i>MS2 class-based TB + scoreboard working</i>	<i>Patrick, Pratibha</i>	<i>04/30/2026</i>	<i>—</i>

<i>MS3 CR + coverage targets hit</i>	<i>Patrick, Pratibha</i>	<i>05/07/2026</i>	<i>—</i>
<i>MS4 UVM env runs base test clean</i>	<i>Patrick, Pratibha</i>	<i>05/14/2026</i>	<i>—</i>
<i>MS5 UVM RAL + regression signed off</i>	<i>Patrick, Pratibha</i>	<i>05/28/2026</i>	<i>—</i>
<i>All high-priority bugs closed or deferred</i>	<i>Patrick, Pratibha</i>	<i>05/28/2026</i>	<i>—</i>
<i>Final zip submitted to Canvas</i>	<i>Patrick, Pratibha</i>	<i>05/28/2026</i>	<i>—</i>

8. Resources and Schedule

8.1 Human Resources

The team consists of two engineers sharing all responsibilities. For each milestone, one team member leads RTL/design tasks while the other leads verification/testbench tasks; roles alternate across milestones to ensure both members gain full exposure to every aspect of the project.

Role	Name(s)	Allocation	Responsibilities
<i>Project Lead (shared)</i>	<i>Patrick, Pratibha</i>	<i>0.5 FTE each</i>	<i>Planning, schedule tracking, Canvas submission, instructor interface</i>
<i>Design / RTL</i>	<i>Patrick, Pratibha</i>	<i>shared</i>	<i>RTL maintenance, bug fixes, parameter extensions</i>
<i>Verification Lead</i>	<i>Patrick, Pratibha</i>	<i>shared</i>	<i>TB architecture, scoreboard, coverage, UVM transition</i>

<i>SVA / Formal (stretch)</i>	<i>Patrick, Pratibha</i>	<i>shared</i>	<i>Assertion authoring, coverage bind</i>
<i>Documentation</i>	<i>Patrick, Pratibha</i>	<i>shared</i>	<i>HLDS, VP, milestone READMEs, final report</i>

Per-Milestone Task Split

Task ownership rotates each milestone so both members cover the full workflow. Shared tasks (review, debug, integration) are owned jointly.

<i>Milestone</i>	<i>RTL / Design Owner</i>	<i>Verification / TB Owner</i>	<i>Shared</i>
<i>MS1</i>	<i>Patrick (initial RTL, defs)</i>	<i>Pratibha (trad TB, tasks)</i>	<i>Git setup, directory structure, review</i>
<i>MS2</i>	<i>Pratibha (RTL tweaks, SV constructs)</i>	<i>Patrick (class-based TB, scoreboard)</i>	<i>Debug, integration</i>
<i>MS3</i>	<i>Patrick (parameter extension, coverage RTL)</i>	<i>Pratibha (CR tests, covergroups, SVA)</i>	<i>Coverage closure</i>
<i>MS4</i>	<i>Pratibha (RTL freeze, UVM bind)</i>	<i>Patrick (UVM agent, env, base test)</i>	<i>UVM bring-up debug</i>
<i>MS5</i>	<i>Patrick (RAL-ready RTL)</i>	<i>Pratibha (UVM RAL, layered seqs, regression)</i>	<i>Coverage closure, final report</i>
<i>FINAL</i>	<i>Both — consolidation</i>	<i>Both — final regression & packaging</i>	<i>Canvas submission</i>

8.2 Hardware Resources

<i>Resource</i>	<i>Specification</i>	<i>Quantity</i>	<i>Availability</i>
------------------------	-----------------------------	------------------------	----------------------------

<i>Simulation Compute</i>	<i>PSU ECE lab workstations or personal laptops (≥ 8 GB RAM)</i>	<i>2</i>	<i>[X] Available</i>
<i>Shared Storage</i>	<i>GitHub repository for all code, docs, and transcripts</i>	<i>1 repo</i>	<i>[X] Available</i>
<i>Local Disk</i>	<i>≥ 5 GB per milestone workspace for waveforms/coverage DBs</i>	<i>as needed</i>	<i>[X] Available</i>
<i>Emulation / FPGA</i>	<i>Not required for this project</i>	<i>—</i>	<i>[] N/A</i>

8.3 Schedule

Schedule aligns with the sign-off dates in Section 7.3. Each milestone's verification work begins the day after the previous milestone's sign-off.

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
<i>M1</i>	<i>MS1 – Design Spec, VP, RTL, traditional TB</i>	<i>04/14/2026</i>	<i>04/20/2026</i>	<i>Both</i>	<i>[] Open</i>
<i>M2</i>	<i>MS2 – Class-based TB + scoreboard</i>	<i>04/21/2026</i>	<i>04/30/2026</i>	<i>Both</i>	<i>[] Open</i>
<i>M3</i>	<i>MS3 – CR + functional coverage + SVA</i>	<i>05/01/2026</i>	<i>05/07/2026</i>	<i>Both</i>	<i>[] Open</i>
<i>M4</i>	<i>MS4 – UVM environment (basic)</i>	<i>05/08/2026</i>	<i>05/14/2026</i>	<i>Both</i>	<i>[] Open</i>
<i>M5</i>	<i>MS5 – UVM RAL + coverage closure</i>	<i>05/15/2026</i>	<i>05/24/2026</i>	<i>Both</i>	<i>[] Open</i>
<i>M6</i>	<i>Final regression run across 50+ seeds</i>	<i>05/25/2026</i>	<i>05/26/2026</i>	<i>Both</i>	<i>[] Open</i>

M7	Final report + waveform snapshots	05/26/2026	05/27/2026	Both	[] Open
M8	Canvas submission (ece592s26_<team>_final_project.zip)	05/28/2026	05/28/2026	Both	[] Open

9. Risk and Mitigation

#	Risk Description	Likelihood	Impact	Mitigation Plan
R1	RTL bugs in MS1 baseline (write latch clear, \$urandom seed) delay MS2 scoreboard bring-up	High	High	Fix identified bugs (see review notes) before MS2 start; add scoreboard-driven regression as early smoke test
R2	Coverage closure not achieved within schedule	Medium	High	Add coverage-driven constrained-random tests; schedule coverage reviews at mid-MS3 and mid-MS5; waiver process for unreachable bins
R3	VCS license or lab access unavailability	Low	High	Both team members use PSU remote lab / ECE server SSH; schedule long coverage regressions off-peak to avoid license contention
R4	Team member unavailable (illness, conflict)	Medium	Medium	Both members cross-trained; all commits reviewed by the other; documentation kept current; tasks rotate each MS

R5	<i>UVM bring-up (MS4) slips due to methodology learning curve</i>	<i>Medium</i>	<i>Medium</i>	<i>Start UVM skeleton in parallel with MS3 CR work; reuse class-based TB driver/monitor logic inside UVM components</i>
R6	<i>Parameter extension (DATA_WIDTH=64) reveals latent RTL bugs</i>	<i>Low</i>	<i>Medium</i>	<i>Limit FV-015 to MS3 regression; raise as separate bugs rather than blocking sign-off if issues found</i>
R7	<i>Scope creep from optional features (formal, MEM_INIT, AxPROT semantics)</i>	<i>Medium</i>	<i>Low</i>	<i>Keep optional items explicitly marked Low priority in Section 2; defer to FINAL milestone if time permits</i>
R8	<i>Git merge conflicts between team members</i>	<i>Medium</i>	<i>Low</i>	<i>Use feature branches; one PR reviewer required before merge; separate directories per MS reduces overlap</i>

10. Document Revision History

<i>Revision</i>	<i>Date</i>	<i>Author</i>	<i>Description of Change</i>	<i>Approved By</i>
<i>0.1</i>	<i>2026-04-19</i>	<i>Patrick, Pratibha</i>	<i>Initial draft for internal review</i>	<i>—</i>
<i>1.0</i>	<i>2026-04-20</i>	<i>Patrick, Pratibha</i>	<i>Baselined for MS1 submission</i>	<i>—</i>
<i>—</i>	<i>[TBD]</i>	<i>Patrick, Pratibha</i>	<i>Updates per milestone as project evolves</i>	<i>[TBD]</i>

